

软工真题-期末题

注：答案来自：自己写，学长笔记，购买的习题集配套答案

软工真题-期末题

期末A卷

一 名词解释

二 解答题

🍏appStore& &googlePlay

解答: OCP + 迪米特法则 + DonotRepeat

改进1:

使用策略模式

使用工厂模式

影片出租店程序

解答：迪米特法则

栈设计的合理性

解答：LSP

设计雇员类

解答：LSP

判断内聚耦合

1. P227课后题

解答：印记耦合 + 通信内聚

2. 判断耦合并修改

解答：控制耦合

改进：拆成数据耦合和印记耦合

逆向写ATM需求

✖设计测试用例

解答

人机交互

解答：

个人是所得税系统

解答：决策表

期末B卷

名词解释

解答题

图书馆借书类

解答：违反OCP开闭原则和单一职责原则

修改1-用接口

修改2 -用抽象类

Sales类getTotal()-迪米特

解答：通信内聚

正方形-长方形-LSP

解答：LSP

前50名成绩-- 内容耦合

解答：内容耦合

ATM 用例图&顺序图

✖ 设计测试用例和测试代码

解答: 见2013年真题解答

人机交互原则 (略)

决策表优化--求每月天数

解答

质量保障活动 (略)

内聚性判断

解答: 时间内聚

图书馆用例图-建模过程

解答:

人机交互原则略)

黑盒白盒各自优缺点

白盒测试方法

16期末回忆

写在前面

文档版本历史

试卷内容

第一大题 (15分)

第二大题 (10分)

第三大题 (10分)

第四大题 (15分)

第五大题 (10分)

第六大题 (10分)

第七大题 (10分)

第八大题 (10分)

第九大题 (10分)

第十大题 (10分)

2005年

简答题

软件成熟度模型

解答

软件配置管理 + 变更源

解答:

🇨🇳 四个基本变更源

解释几种测试方法

解答

2013

解答

✖ 有理数类的实现与测试 (JUnit)

正方形类-长方形

解答

2014

解答题

描述软件体系结构分层风格 (略)

建立分析类图---创建路单

解答: 来自学长 (P100)

分析代码质量不高的地方并改进 -- Department类

🇨🇳 <简单工厂>改进

2015

名词解释:

解答题

图示面向对象体系结构风格, 优缺点 (略)

建立分析类图 -- 自驾游路线设计

解答: 见「软件工程强化与冲刺」

代码质量不高的地方并改进 -- init() 方法

解答:

改进1: 拆分类

改进2: 拆分方法

修改不足 -- 多个compare

解答: 用策略模式

2016

质量保障的三种手段

判断需求类型

解答:

ATM取款用例

解答:

解答二

违反原则-改进代码 --- employee类

解答

用契约式/防御式 改写代码 --- Deposit类

解答

2017

名词解释

软件工程

持续集成

需求题

需求的三个层次

从三个层次出发, 描述图书管理系统中的需求

体系结构 -- 大小写转换

解答: P150

类的职责与封装 -- person类

解答

面向对象 -- 计算个人所得税

解答: 违反迪米特

面向对象 -- 网络选课系统

解答

消除代码重复 -- getTotalSum

解答1

解答2

测试题

什么是黑盒测试

黑盒测试的方法

2018

问答题

科学与工程的区别

集成测试, 单元测试, 集成测试与单元测试用例的区别

功能性需求与非功能性需求

四个软件体系结构视角

质量模型的可用性

✗ 房屋评估系统的合理性

解答

✗ 计算税费

解答

2013备用卷

面向对象 -- 计算点之间的距离与方向

解答：没有集中信息与行为

✗ 智能家居系统

解答

2019

简答题

生命周期模型

功能需求与非功能需求

软件开发不同阶段的质量保证手段

体系结构的四个视角

黑盒白盒差异

迪米特法则 --- sales类

解答： P233

设计模式 --- 飞机类

解答

附录 -- 遗留未解决的问题

2017年个人所得税问题

解答：违反迪米特

几个软件测试题代码

解答

2013年备用 按钮-灯泡-门

2018房屋评估系统

2018 计算税费问题

⚠️：目前还没掌握

🇨🇳：很重要但没掌握

✅：重要但目前掌握

☕：不那么重要，喝杯Java休息一下

✗：还没搞懂

期末A卷

一 名词解释

1. 软件工程

- 应用系统的、规范的、可量化的方法来开发、运行和维护软件
- 对其中各种方法的研究

2. ⚠️ 软件演化生命周期模型

- 初始开发 ---> 演化 ---> 服务 ---> 逐步淘汰 ----> 停止

3. ⚠️ 螺旋模型

- 版本一：按照风险解决的方式来组织软件开发活动
- 版本二：
 - 是一种周期性的方法进行系统开发
 - 有许多"中间版本"
 - 每个周期都包括续期定义、风险分析、工程实现、和评审 四个阶段

二 解答题

🍏 appStore & googlePlay

Sakuragi 开发了一个手机应用，准备投放到 Apple App Store 和 Google Play 市场中去，下面是他应用的部分关于应用描述的代码，请分析其设计是否合理，是否违反某些设计原则，是否能够应用某种设计模式来重构。

- 1) 指出违反的原则，请解释该原则，并给出修改后的代码
- 2) 解释该设计模式，写出应用该设计模式后的代码

```
public class Application{
    private String applicationName;
    private float averageRate;
    private ArrayList<NewFeature> newFeatureItems = new ArrayList<NewFeature>();

    public static String getDescriptionForIOS(){
        StringBuffer result = new StringBuffer();
        result.append("This is "+ this.applicationName + " for IOS platform\n");
        for(int i= 0;i<newFeatureItems.size();i++){
            result.append(newFeatureItems.get(i).getDescription());
        }
        result.append("Average Rate from APP Store\n");
        result.append(String.valueOf(averageRate));
        return result.toString();
    }

    public static String getDescriptionForAndroid(){
        StringBuffer result = new StringBuffer();
        result.append("This is "+ this.applicationName + " for Android platform\n");
        for(int i= 0;i<newFeatureItems.size();i++){
            result.append(newFeatureItems.get(i).getDescription());
        }
        result.append("Average Rate from Google Play\n");
        result.append(String.valueOf(averageRate));
        return result.toString();
    }
}
```

解答: OCP + 迪米特法则 + DonotRepeat

违反了OCP原则：对扩展开放，对修改关闭（当IOS平台或者安卓平台其中一方发生变更时会导致另一方也被修改(因为在同一个类中，无无论方法是否变更，都算修改)。所以应将两个逻辑上并立的代码分开封装，并抽象一个统一的接口口）

违反了DO not repeat：有相同逻辑代码

违反了迪米特法则：一个软件实体应最少地与其他软件实体发生相互作用（for循环中出现隐式方法，所以应该使用迭代器访问集合对象，减少与其他类的交互）。改进后的代码如下

改进1:

```
/*
 * 违反了OCP和迪米特法则
 * 修改
 */
public class Application{
    private String applicationName;
    private float averageRate;
    private ArrayList<NewFeature> newFeatureItems = new ArrayList<NewFeature>();

    private Phone phone;
    Phone = new AndriodPhone();
    //Phone = new Iphone();

    public static String getDescription(){
        StringBuffer result = new StringBuffer();
        result.append("This is "+ this.applicationName + " for" +
phone.getPlatformName()+ " platform\n");

        for(int i = 0;i<newFeatureItems.size();i++){
            result.append(newFeatureItems.get(i).getDescription());
        }
        // for循环隐式访问方法，应该使用迭代器
        Iterator<NewFeature> it = newFeature.iterator();
        while(it.hasNext()){
            result.append(it.next().getDescription());
        }

        result.append("Average Rate from "+ phone.getSourceName()+"\n");
        result.append(String.valueOf(avarageRate));
        return result.toString();
    }
}

public interface Phone{
    public String getSourceName();
    public String getPlatformName();
}
```

```

}
public class AndriodPhone implements Phone{
    private String sourceName = "Google Play";
    private String platformName = "Andriod";
    public String getSourceName(){
        return this.sourceName;
    }
    public String getPlatformName(){
        return this.platformName;
    }
}
public class Iphone implements Phone{
    private String sourceName = "APP Store";
    private String platformName = "IOS";
    public String getSourceName(){
        return this.sourceName;
    }
    public String getPlatformName(){
        return this.platformName;
    }
}
}

```

使用策略模式

```

/*
 * 使用策略模式
 */

public class TestCase{
    public void test(){
        Application andriodApp = new Application();
        andriodApp.setApplicationName("淘宝");
        andriodApp.setAverageRate(0.7);
        andriodApp.setNewFeatureItems(new ArrayList<NewFeature>());

        Device andriod = new Andriod();
        andriodApp.setDevice(andriod);
        andriodApp.getDescription();

    }
}

public class Application{
    private String appicationName;
    private float averageRate;
    private ArrayList<NewFeature> newFeatureItems = new ArrayList<NewFeature>();

    private Device device;
}

```

```

    public void setDevice(Device device){
        this.device = device;
    }

    public String getSourceName(){
        device.getSourceName();
    }
    public String getPlatformName(){
        device.getPlatformName();
    }
    public String getDescription(){
        StringBuffer result = new StringBuffer();
        result.append("This is " + this.applicationName + " for" + getPlatformName() + "
platform\n");
        // for循环隐式访问方法, 应该使用迭代器
        Iterator<NewFeature> it = newFeature.iterator();
        while(it.hasNext()){
            result.append(it.next().getDescription());
        }
        result.append("Average Rate from " + getSourceName()+ "\n");
        result.append(String.valueOf(avarageRate));
        return result.toString();
    }
}

public interface Device{
    public String getSourceName();
    public String getPlatformName();
}

public class Andriod implements Device{
    public String getSourceName(){
        return "Google Play";
    }
    public String getPlatformName(){
        return "Andriod";
    }
}

public class IOS implements Device{
    public String getSourceName(){
        return "App Store";
    }
    public String getPlatformName(){
        return "IOS";
    }
}
}

```

使用工厂模式


```

/*
使用工厂模式
*/

interface App{
    public String getDescription();
}
public class IOS implements App{
    public String getDescription(){

    }
}
public class Andriod implements App{
    public String getDescription(){

    }
}
interface Factory{
    public App createApp();
}
public class iosApp implements Factory{
    public App createApp(){
        return new IOS();
    }
}
public class andriodApp implements Factory{
    public App createApp(){
        return new Andriod();
    }
}
}

```

影片出租店程序

例【3】 Rukawa同学开发一个影片出租店用的程序，其中需要计算客户的积分，如果电影是新发布的电影并且租用的时间超过1天，则可以得到2点积分，否则是1点积分。

(1)画出下列代码设计的顺序图

(2)指出是否违反了某些设计原则，并解释。

(3)对其代码进行修改，写出其修改后的代码并画出修改的顺序图

```

public class Customer{
    Rental rental;
    int getNewRentPoint(){
        Movie m = rental.getMovieRented();
        if((m.getPriceCode == Movie.NEW_RELEASE)&& rental.getDaysRented(>1){
            return 2;
        }else {
            return 1;
        }
    }
}

```

```

    }
}
}
public class Rental{
    private int daysRented;
    private Movie movieRented;

    private int getDaysRented(){
        return daysRented;
    }
    public Movie getMovieRented{
        return movieRented;
    }
}
public class Movie {
    private int priceCode;
    public static final int CHILDRENS = 2;
    public static final int REGUALR = 20;
    public static final int NEW_RELEASE = 1;

    public int getPriceCode{
        return priceCode;
    }
}
}

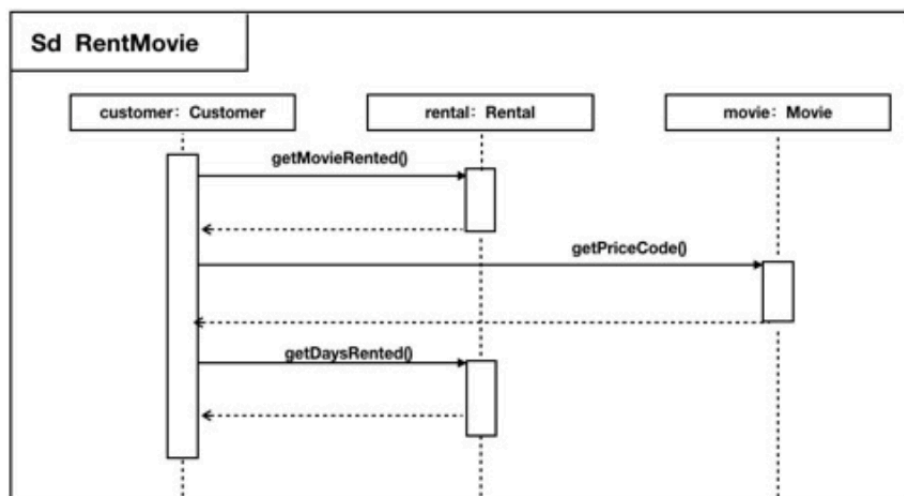
```

解答：迪米特法则

违反了迪米特法则：一个软件实体应尽可能少与其他软件实体发生相互作用

代码顺序图

(1)代码设计图如下



```

/*
违反了迪米特法则
*/

public class Customer{
    Rental rental;
    int getNewRentPoint(){
        rental.getMovieRentPoint();
    }
}

public class Rental{
    private int daysRented;
    private Movie movieRented;

    public int getDaysRented(){
        return daysRented;
    }

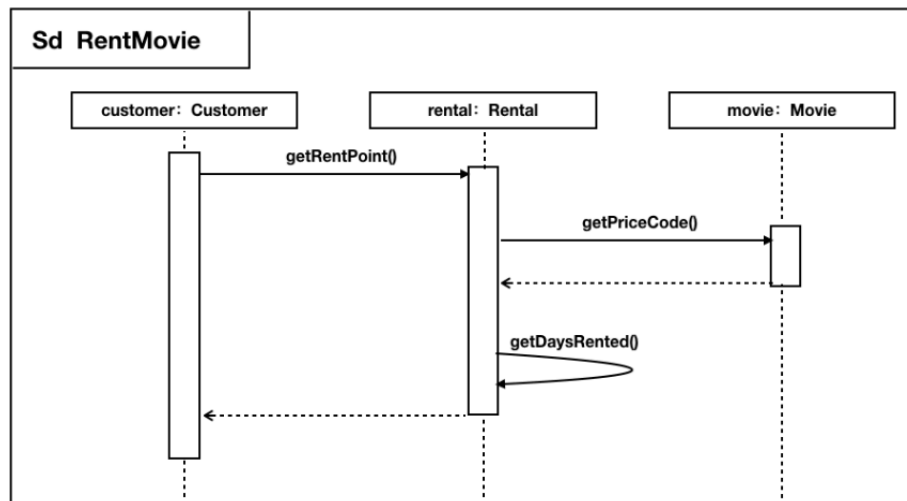
    public int getMovieRentPoint{
        if(movieRented.getPriceCode == Movie.NEW_RELEASE && this.getDaysRented()){
            return 2;
        }else{
            return 1;
        }
    }
}

public class Movie {
    private int priceCode;
    public static final int CHILDRENS = 2;
    public static final int REGUALR = 20;
    public static final int NEW_RELEASE = 1;

    public int getPriceCode{
        return priceCode;
    }
}

```

修改后的代码顺序图



1. eclipse 中人机交互优点和体现的人机交互原则

- **简洁**: 不要使用太大的菜单, 不要在一个窗口表现过多的信息类别
- **一致性**: 依据精神模型, 不要让按钮的位置不一样
- **低出错率设计**: 提供简洁的指导帮助用户消除错误, 用户输入错误信息, 系统会标注出来, 而不是让用户重写整段代码
- **易记性**: 减少用户的记忆负担, 使用逐层递进的范式展示信息, 并且小功能使用直截了当的图像快捷方式
 - 减少短期记忆负担
 - 逐层展示信息
 - 使用直观的快捷方式
 - 设置有意义的默认值

栈设计的合理性

数据结构栈有四个功能: 压栈、弹栈、得到栈的大小、得到栈是否为空。Akagi 同学使用继承如下设计了栈。

```

public class MyStack extends Vector{
    public void push(Object element){
        insertElementAt(element,0);
    }
    public Object pop(){
        Object result = firstElement();
        removeElementAt(0);
        return result;
    }
}

```

解答: LSP

不符合LSP: 子类可以替换父类并起到相同的作用

(1. Vector 中的size和isEmpty 栈中并没有, 这样修改了父类的接口

2. 只为了复用而不为了组织类型差异的继承用法通常是不符合里氏替换原则的，并建议使用组合代替继承)

```
public class MyStack{
    Vector vector = new Vector();
    public void push(Object element){
        vector.insertElement(element,0);
    }
    public Object pop(){
        Object result = vector.firstElement();
        vector.removeElement(0);
        return result;
    }
}
```

设计雇员类

Kogure 同学在设计雇员类的时候，如下设计:

```
public class person{
    private string name;
    public string getName(){
        return name;
    }
}
public class employee extends Person(){
}
```

解答：LSP

子类表达一个特殊类型，而不是一个扮演角色，违反LSP

```
//组合
public class person{
    private string name;
    public string getName(){
        return name;
    }
}
public class employee{
    public Person person = new Person();
    public string getName(){
        person.getName()
    }
}
```

```
//接口
public interface Person{
    public string getName();
}
public class employee implements Person{
    public string name;
    public string getName(){

    }
}
```

判断内聚耦合

1. P227课后题

```
validate_checkout_request(input_form i){
    if(!valid_string(i.name)){
        error_message("invalid name");
    }
    if(!valid_month(i.date)){
        error_message('invalid month');
    }
}
int valid_month(date d){
    return d.month>1 && d.month <=12;
}
```

解答：印记耦合 + 通信内聚

两个函数之间印记耦合（PPT答案里说是数据耦合），第一个函数内部通信内聚

2. 判断耦合并修改

```
void validdata_checkout_request(input_form i){
    if(!valid(i.name,STRING)){
        error_message("invalid name");
    }
    if(!valid(i.data,DATE)){
        error_message("invalid month");
    }
}
int valid(string s, int type){
    switch(type){
        case STRING:return strlen(s) < MAX_STRING_SIZE;
        case DATE:
            date d = parse_date(s);
            return d.month >=1 && d.month <=12;
    }
}
```

```
}
```

解答：控制耦合

改进：拆成数据耦合和印记耦合

```
void validdata_checkout_request(input_form i){
    if(!valid_string(i.name)){
        error_message("invalid name");
    }
    if(!valid_date(i.date,)){
        error_message("invalid month");
    }
}
int valid_string(string s){

}
int valid_date(string s){

}
```

逆向写ATM需求

假设需要你逆向常见 ATM 机的需求，(1)例说明 ATM 机的业务需求、用户需求以及系统级需求，描述 ATM 机的功能需求[提示:概括存款、取款、查询和转账功能]；(2)除了功能需求之外，还有哪些需求需要定义?试一一列举。

- 业务需求：提供在线转账服务
- 用户需求：允许用户查询余额
- 系统需求：在用户点击退卡时，系统应将银行卡从卡槽中推出
- 功能：系统应提供存款服务
- 性能：所用用户操作在3秒内得到响应
- 质量属性：在发生网络故障时，系统不能故障
- 对外接口：系统与数据库系统的接口
 - 异常处理要求：如果方法抛出异常，数据回滚为未操作前的状态
- 约束：用java开发
- 数据：系统要存储一年的操作记录

需求分类：功能，性能，质量属性，对外接口，约束

✗ 设计测试用例

Kawata 同学希望测试 Akagi 同学设计的类

```
Public class MyStackextendsVector{
    Public void push(Objectelement){
        insertElementAt(element,0);
    }
    Public Object pop(){
        Object result=firstElement();
        removeElementAt(0);
        Return result;
    }
}
```

- 1)完成功能测试的测试用例的设计，说明思路。
- 2)给出相应的测试代码。

解答

参考答案（不太靠谱）

(1)

ID	输入	预期输出
1	push("a"),push("b")	a
2	push(1),push(2)	1

(2)class Test {

```
Public static void main() {
    Mystack s  = new Mystack();
    s.push("a");
    s.push("b");
    s.pop();
    s.show();
}
```

}

}

人机交互

Sawakita 同学下载了 Eclipse 软件。请支持至少 3 条该软件在人机交互方面的有些优点，分析它们体现了哪些人机交互的原则？



解答：

- 简介设计：不要再一个窗口展示太多信息
- 一致性设计：按钮的相对位置相同
- 低出错率设计：使用简洁的方式指导用户，在用户出错的时候，指出出错的一行
- 易记性设计：
 - 减少短期记忆负担，代码提示
 - 使用逐层递进的方式展示信息，：多级目录展示
 - 使用直观的快捷方式，使用直观快捷方式
 - 设置有意义的默认值：默认创建 main函数

个人是所得税系统

Fujima 开发一个个人所得税系统，代码如下。其中缴税的规则是：

- 1、For the first \$10,000 of income, the tax is 10%
- 2、For the next \$10,000 of income above \$10,000, the tax is 12 percent
- 3、For the next \$10,000 of income above \$20,000, the tax is 15 percent
- 4、For the next \$10,000 of income above \$30,000, the tax is 18 percent
- 5、For any income above \$40,000, the tax is 20 percent

```
/*
    个人所得说系统
    用决策表来优化
*/

/*
    题目：
        小于 10000 tax is 10%
        大于 10000 小于 20000 的部分 12%
        大于 20000 小于 30000 的部分 15%
        大于 30000 小于 40000 的部分 18%
        大于 40000 的部分 20%
*/
class Main{
    public int calculateTax(int taxable_income){
```

```

int tax = 0;
if(taxable_income == 0){
    //goto Exit;
    return 0;
}
if(taxable_income > 10000){
    tax = tax + 1000; //10000以下的那部分税
}else{
    tax = tax + taxable_income * 0.1;
    //goto Exit;
    return tax;
}
if(taxable_income > 20000){
    tax = tax + 1200;
}else { //小于20000
    tax += (taxable_income-10000)*0.12;
    return tax;
}
if(taxable_income > 30000){
    tax = tax + 1500;
}else { //小于 30000
    tax += (taxable_income-20000)*0.15;
    return tax;
}
if(taxable_income > 40000){
    tax = tax + 1800;
    tax += (taxable_income-40000)*0.2;
}else {
    tax += (taxable_income-30000)*0.18;
    return tax;
}
return tax;
}
}

```

解答：决策表

```

/*
    修改
*/

int[] percent = {10%,12%,15%,18%,20%};
int[] bracket = {0,10000,20000,30000,40000};
int[] base = {0,1000,1200,2200,3700,5500}

public int calculateTax(int taxable_income){
    int level = 1;

```

```

    for(int i=0;i<5;i++){
        if(taxable_income > bracket[i]){
            level++;
        }
    }
    int tax = base[level] +(taxable_income - bracket[level])*percent[level];
    return tax;
}

/*
    参考表驱动编程 P307
*/

int[] prePoint = {1000,2000,5000};
int[] postPoint = {1000,2000,5000};
int[] levelArray = {1,2,3};
public int calculateLavel(int prePoint ,int postPoint){
    for(int i=0;i<3;i++){
        if(prePoint < prePoint[i] && postPoint >= postPoint[i]){
            return levelArray[i];
        }
    }
}
}

```

期末B卷

名词解释

1. 软件工程 *
2. 软件验证与确认
 - 验证：检查开发者是否正确地使用技术建立系统，确保系统能够在预期的环境正按照技术要求正确地运行。
 - 确认：检查开发者是否建立了正确的系统，确保最终产品符合规格。
 - （验证和确认是测试的两个层面，前者指软件过程，后者指交付结果。验证多是针对设计而言的，确认是对应需求而言的。验证是看期望的结果经过指定的过程和参数能否实现，确认是要看到底有哪些期望，是否与用户需求相符）
3. 增量开发模型和迭代开发模型
 - 需求驱动
 - 增量迭代模型在项目开始时，通过系统需求开发和核心体系结构设计活动完成项目对前景和范围的界定，然后再将后续开发活动组织为多个迭代，并行的瀑布式开发活动。

解答题

图书馆借书类

图书管理系统中有多个借阅者角色。本科生、研究生和教师。所有借阅者都可以借阅图书。教师借阅图书的行为和本科生,研究生略有不同时。当教师希望借阅的某种图书被借空时,系统将自动通知借阅者归还图书,本科生只可借阅普通图书,最多可同时借阅 5 本;研究生可以最多可同时借阅 10 本;老师可以借阅 20 本。Hyoga 同学熟悉结构化编程,给出了如下设计。请根据以上借阅图书相关的功能性需求和面向对象的思想,指出 Hyoga 设计的问题,画出关于借阅者的设计类图,并且写出各个类和借阅相关的属性和方法的定义(不用实现)。

```
public class Borrower{
    int MAX_FOR_BACHELOR=5;
    int MAX_FOR_MASTER=10;
    int MAX_FOR_TEACHER=20;

    public void borrowBookForBorrower(int i){
        if(i==0) borrowBookForBachelor();
        else if(i==1) borrowBookForMaster();
        else if(i==2) borrowBookForTeacher();
    }

    private void borrowBookForBachelor(){
        borrowBook();
    }

    private void borrowBookForMaster(){
        borrowBook();
    }

    private void borrowBookForTeacher(){
        borrowBook();
        notifyReturnBook();
    }

    private void borrowBook(){
        //借书
    }

    private void notifyReturnBook(){
        //通知还书
    }
}
```

解答：违反OCP开闭原则和单一职责原则

修改1-用接口

```
public interface Borrower{
    public void borrow();
}

public class BachelorBorrower implements Borrower{
    int MAX_FOR_BACHELOR=5;
    public void borrow(){
        //借书
    }
}

public class MasterBorrower implements Borrower{
    int MAX_FOR_MASTER=10;
    public void borrow(){
        //借书
    }
}

public class TeacherBorrower implements Borrower{
    int MAX_FOR_TEACHER=20;
    public void borrow(){
        //借书
        notifyReturnBook();
    }

    private void notifyReturnBook(){
        //通知还书
    }
}
```

修改2 -用抽象类

```

public abstract class Borrower{
    public abstract void borrow();

    public void borrowBook(){
        //借书
    }
}

public class BachelorBorrower extends Borrower{
    int MAX_FOR_BACHELOR=5;
    public void borrow(){
        borrowBook();
    }
}

public class MasterBorrower extends Borrower{
    int MAX_FOR_MASTER=10;
    public void borrow(){
        borrowBook();
    }
}

public class TeacherBorrower extends Borrower{
    int MAX_FOR_TEACHER=20;
    public void borrow(){
        borrowBook();
        notifyReturnBook();
    }

    private void notifyReturnBook(){
        //通知还书
    }
}

```

Sales类getTotal()-迪米特

小计每一购物项金额的方法放在 Sales 类中

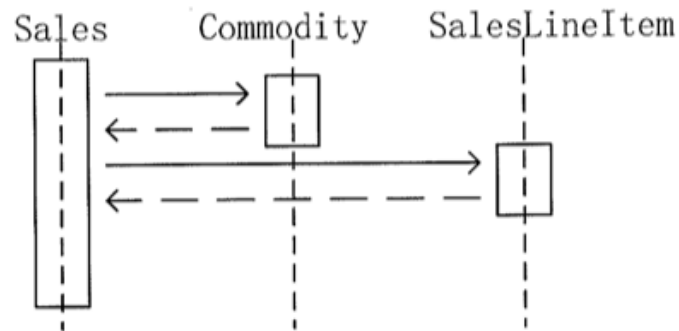
```

class Sales{
    ...
    getSubtotal(int CommodityID){
        //1.根据 CommodityID 找到商品 Commodity 对象，获得商品的价格。
        //2.根据 CommodityID 找到销售商品项 SalesLineItem 对象,再找到商品购买的数量。
        //3.计算小计。
    }
}

```

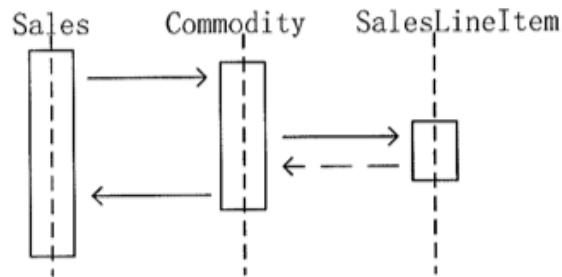
- 1) 请画出下列代码设计的顺序图。（系统已有 Sales 对象、Commodity 对象、SalesLineItem 对象）
- 2) getSubtotal 内聚性如何？是否违反某些设计原则，解释这些原则。
- 3) 画出修改之后的顺序图。

解答：通信内聚



2) 通信内聚：违反迪米特法则，解释参考教材，不能有跨层或者多余的关系

3)



```

class Sales{
    public int getSubtotal(int commodityId){
        SalesLineItem salesLineItem = SalesLineItemMap.get(commodityId);
        return salesLineItem.getSubtotal();
    }
}
class SalesLineItem{
    Commodity commodity;
    int nums;
    public getSubtotal(){
        return commodity.getPrice * nums;
    }
}
class Commodity{
    int price;

    public int getPrice() {
        return price;
    }

    public void setPrice(int price) {
        this.price = price;
    }
}
  
```

正方形-长方形-LSP

1) 正方形类可否直接继承长方形类?

2) 如果合理，请解释其合理性。如果违反，请解释该原则，并修改。

```
/*
    长方形和正方形
*/
class Rectangle{
    int length;
    int width;

    public int area(){
        return length*width;
    }

    public int getLength() {
        return length;
    }

    public void setLength(int length) {
        this.length = length;
    }

    public int getWidth() {
        return width;
    }

    public void setWidth(int width) {
        this.width = width;
    }
}
```

解答：LSP

违反里氏替换法则：子类必须能够提换基类并起到相同的作用

```
/*
    组合代替继承
*/

class Square{
    Rectangle rectangle;
    int edge;
    public Square(int edge){
        rectangle = new Rectangle(edge,edge);
    }
    public void setEdge(int edge) {
        this.rectangle.setWidth(edge);
        this.rectangle.setWidth(edge);
    }
}
```

```

    public int area(){
        return rectangle.area();
    }
}

```

前50名成绩-- 内容耦合

辅导员需要计算全年级前 50 名的平均学成绩。

```

public class Grade{
    public float averageGradefroTop50(ArrayList<Student>allStudent){
        ArrayList<Student> sortedStudent = allStudent;
        int totalGrade = 0;
        for(int i=0;i<50;i++){
            totalGrade += sortedStudent.get(i).getGrade();
        }
        double averageGrade = totalGrade/50.0;
        return allStudent;
    }
}

```

解答：内容耦合

修改

```

/*
    方法一： 利用组合
*/
public class Grade{
    ArrayList<Student>sortedStudet;

    public ArrayList<Student> getSortedStudet() {
        return sortedStudet;
    }

    public void setSortedStudet(ArrayList<Student> sortedStudet) {
        this.sortedStudet = sortedStudet;
    }

    public float averageGradefroTop50(){
        int totalGrade = 0;
        for(int i=0;i<50;i++){
            totalGrade += this.sortedStudent.get(i).getGrade();
        }
        double averageGrade = totalGrade/50.0;
        return averageGrade;
    }
}
/*

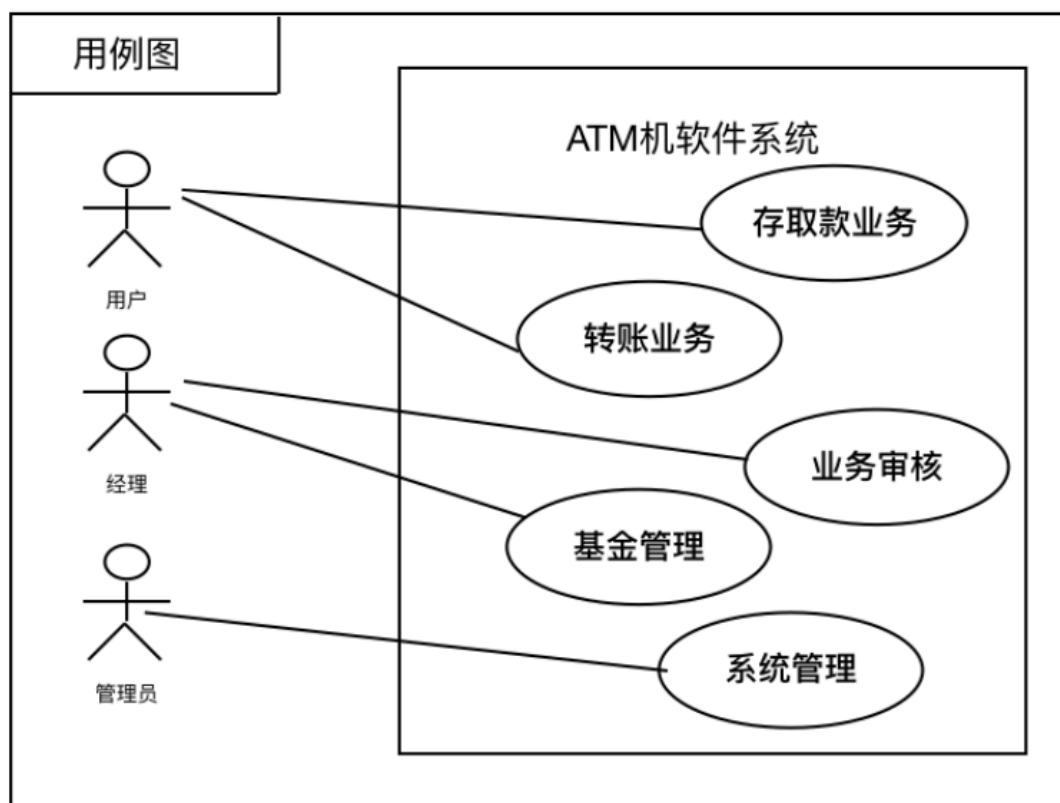
```

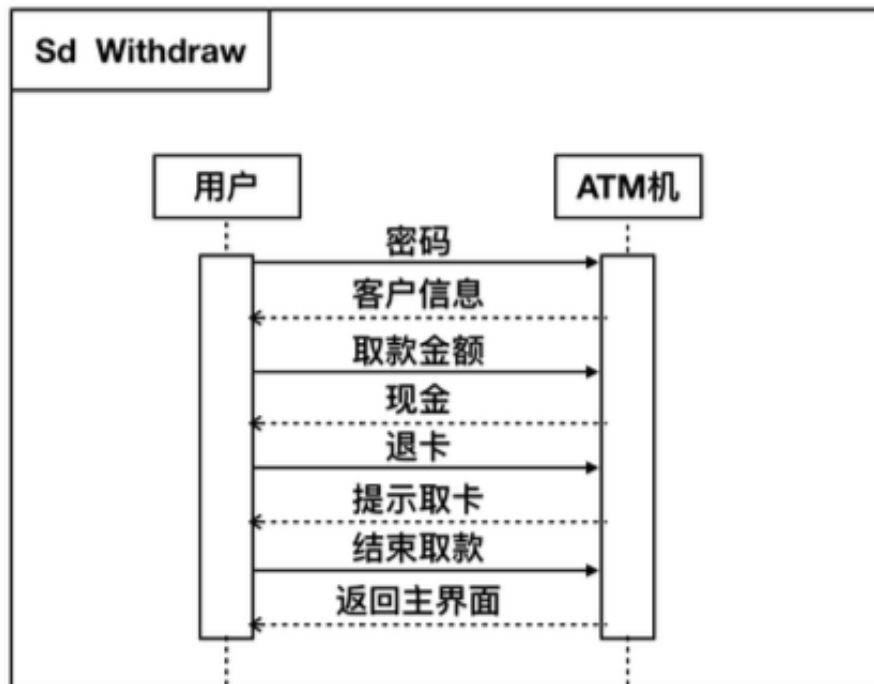

另一种方法：迭代器

```
*/  
public class Grade{  
    public float averageGradefroTop50(ArrayList<Student>allStudent){  
        ArrayList<Student> sortedStudent = allStudent;  
        Iterator<Student> iterator = sortedStudent.iterator();  
        int totalGrade = 0;  
        for(int i=0;i<50;i++){  
            Student student = iterator.hasNext();  
            totalGrade += student.getGrade();  
        }  
        double averageGrade = totalGrade/50.0;  
        return allStudent;  
    }  
}
```

ATM 用例图&顺序图

画出常见 ATM 机软件系统的用例图。选择其中一个用例，画出其系统顺序图。





✗ 设计测试用例和测试代码

Kawata 同学希望测试自己实现的有理数类

```
public class MyRationalNum{
    ...
    public MyRationalNumadd(MyRationalNumi){
    ...
    }
    public MyRationalNumdividedBy(MyRationalNumi){
    ...
    }
}
```

- 1) 利用黑盒测试的方法完成功能测试的测试用例的设计，说明思路；
- 2) 给出相应的测试代码。

解答: 见2013年真题解答

参考答案（感觉不太靠谱）

1)

ID	输入	预期输出
1	0 1 1	有理数
2	-1 1 1	有理数
3	1 1 2	有理数
4	1 1 3	无理数
5	-1 1 3	无理数

2) class Test {

```
    Public static void main [] {  
        myRationalNum R = new myRationalNum();  
        R.add(0);  
        R.divideby(1);  
    }  
    .....  
    .....  
}
```

人机交互原则（略）

分析一款常用的浏览器。请支持至少 3 条该软件在人机交互方面的有些优点，分析它们体现了哪些人机交互的原则？

决策表优化--求每月天数

解答

```
class main{  
    int[] days = {31,28,31,30,31,30,31,31,30,31,30,31}  
    int getDaysofMonth(int month){  
        return days[i-1];  
    }  
}
```

质量保障活动（略）

- 需求开发 需求评审，需求度量
- 体系结构 体系结构评审，集成测试
- 详细设计 详细设计评审，设计度量，集成测试
- 软件构造 代码评审，代码度量，测试（测试驱动、持续集成）
- 软件测试 测试，测试度量

内聚性判断

```

public class A {
    Private:
        FinancialReport fr;
        WeatherData w;
        int totalCount;

    Public:
        A();
        void init();
        ...
        ~A();
}

void init(){
    //初始化财务报告
    fr=new(FinancialReport);
    fr.setRatio(5);
    fr.setYear("2010");

    //初始化当前天气
    w=new(WeatherData);
    w.setCity("NanJing");
    w.setCode("210093");

    //初始化计数器
    totalCount=0;
}

```

解答：时间内聚

改进

```

public class A {
    Private:
        FinancialReport fr;
        WeatherData w;
        int totalCount;

    Public:
        A();
        void initFinancialReport();
        void initWeatherData();
        void initTotalCount();
        ...
        ~A();
}

void initFinancialReport(){
    //初始化财务报告
    fr=new(FinancialReport);
    fr.setRatio(5);
    fr.setYear("2010");
}

void initWeatherData(){
    //初始化当前天气
    w=new(WeatherData);
    w.setCity("NanJing");
    w.setCode("210093");
}

void initTotalCount(){
    //初始化计数器
    totalCount=0;
}

```

图书馆用例图-建模过程

例【2】.为下列描述建立用例模型，要求给出明确的建模过程。

现在需要开发一个简化了的大学图书馆系统，它有几种类型的借书人。包括教职工借书人，研究生借书人和本科生借书人等。借书人的基本信息包括姓名、地址和电话号码等，对于教职工借书人，还要包括诸如办公室地址和电话等信息。对于研究生借书人，还要包括研究项目和导师信息等。对于本科借书人，还要包括项目和所有学分信息等。

图书馆系统要跟踪借出书本信息，当一个借书人捧着一堆书去借书台办理借书手续时，借出这个事件就发生了，随着时间的过去，一个借书人可以多次从图书馆中借书，一次可以借出多本图书。如果借书人想要的书已经被借出，他可以预约，每个预约只针对一个借书人，和一个标题，预约日期，优先权和完成日期等信息需要维护，当借书完成，系统会将这本书与借出联系起来。

借书人根据图书馆的信息来检索书名，同时检索这本书是否可以被借出，如果一本书的所有副本都被借出了，那么借书人可以根据书名预定这本书，当借书人把书拿到借书台的时候，管理员可以为这些书办理归还手续，管理员要跟踪新书到达的情况。

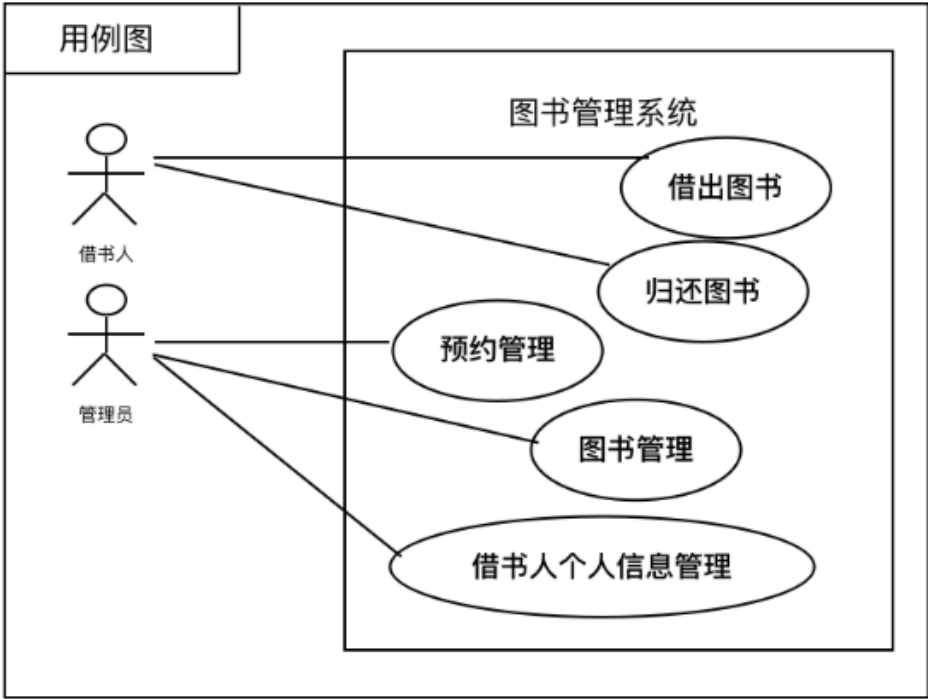
图书的管理者有属于自己的活动，他们要分类打出关于书的标题的表格，还要在线检查所有过期未还的图书也标出来，而且图书馆系统还可以从另外一个大学的数据库中访问和下载借书人的信息。

解答：

①建模过程：

- 1.明确系统的目标：管理图书馆的书本借阅与维护信息
- 2.寻找参与人:教职工借书人、研究生借书人、本科借书人、管理员
- 3.寻找用例:借书人个人信息管理、借出图书、预约管理、归还图书、图书管理
- 4.细化用例:图书管理可分化为:馆内图书管理、借出图书管理

②用例模型



【注】这里虽然找到了4个参与者，但是重读用例文档，可以发现描述文档中对3种借书人没有明显的区分，不好细化用例，所以这里我就直接将三种借书人都归结为借书人。当然，对于这种答案，没有标准答案，只要注意模板细节，符合题意都可以。

人机交互原则略)

黑盒白盒各自优缺点

- 白盒优点：覆盖率高；发现的缺陷较多
- 白盒缺点：测试开销大；不能检验需求规格
- 黑盒优点：测试效率高；可以检验需求规格
- 黑盒缺点：覆盖率低：发现的缺陷少

白盒测试方法

（2）现在有一段对正确性要求很高的程序，它含有多个复杂的分支判断结构，这些分支结构中有一些互相嵌套，有一些顺序执行。你认为应该使用下面哪种白盒测试方法测试该段程序，并给出理由：随机选择、语句覆盖、分支覆盖和路径覆盖（4分）。

采用路径覆盖：确保程序中每条独立的路径都被至少执行一次

注：几种覆盖：

- 语句覆盖：确保被测试对象中的每一行程序都被至少执行一次
- 条件覆盖：确保程序中每个判断的结果都至少满足一次

16期末回忆

写在前面

考试内容摘自小笼包的考场回忆。

以下内容中，您可能会看到：

- 不准确的分数划分（按照当时记下来的内容的确是120分.....原因不明（附加题吗？））
- 模糊的题目描述
- 不准确或错误的记叙
- 因课程大纲改革带来的可能的题目差异

希望您在参考本文档时能多思考而并非填鸭式抱佛脚。

预祝各位考出理想的成绩！

欢迎各位对此文档进行更正和增补。

小笼包 2018.7.9

文档版本历史

时间	作者	备注	版本
2018/7/9	小笼包	创建了本文档	1.0.0

试卷内容

第一大题（15分）

1.什么是软件工程？

- 。 。 。

2.简述演化模型及其优缺点

- 演化模型：早期明确系统的核心需求，建立并交付核心系统。依据用户反馈迭代并行的瀑布是开发活动进行精化 增强系统
- 优点：适用性好； 并行开发缩短了开发时间；渐进交互加强了用户反馈，开发风险降低
- 缺点：项目范围等计划无法早期确定；后期迭代易蜕变为 build-fix 模型

3.简述逆向工程与正向工程的区别，并各用一句话说明其关注点

- 软件开发阶段（正向工程）会书写清晰的文档，留下可读性较好的程序源代码
- 逆向工程可能没有文档，甚至没有源代码；基本原理是抽取软件系统的需求与设计而隐藏实现细

节，然后在需求和设计的层次上描述软件系统，以建立对系统更加准确和清晰的理解

- 正向工程：模型转换为代码
- 逆向工程：代码转换为模型

第二大题（10分）

阅读材料，回答问题：

消费者可以使用支付宝“扫一扫”，输入金额和密码进行支付，也可以让商家扫描用户的付钱码进行支付。
卡包内有优惠券、红包等可以在支付时用于抵现。
(此处应有图，可参考手机支付宝APP的付款页面)

解释用例图的四要素，并画出用例图。

第三大题（10分）

以下为活动“扫描商家的二维码进行付款”的用例的概念类的候选类：

消费者，商家，蚂蚁积分，付款码，付款方式，花呗支付，余额支付，余额宝支付，银行卡支付

请识别概念类之间的关系（依赖、聚合、关联、组合、继承等），识别重要属性，画出概念类图。

第四大题（15分）

1.画出付款模块的物理包图（包括分层和跨网络）（7分）

2.写出付款用例对应展示层和逻辑层交互的接口（登陆、填写支付金额、输入付款密码等），以及逻辑与数据层之间的接口（登陆账户查询、记录付款信息、更新积分信息）（8分）

第五大题（10分）

用花呗支付后，用户积分也会发生改变。请画出详细设计中支付界面对象、支付功能逻辑对象、花呗逻辑对象、用户逻辑对象、支付数据对象、花呗数据对象和用户数据对象之间的关系。

第六大题（10分）

简述如何消除印记耦合。

第七大题（10分）

现在支付宝中的会员等级分为A、B、C三等，如果现在要实现添加一个S级等级，并修改不同等级积分的计算方式（例如，A等级为消费一元积1分，B为二元积一分），应如何实现？请画出类图并解释你的实现方式（需要用到设计模式）。

第八大题（10分）

使用表驱动改写冗长的if-else结构体。

第九大题（10分）

使用黑盒测试的方法，测试支付宝的“更改用户密码”功能，写出输入和预期输出。

第十大题（10分）

观察下面两图，写出界面中体现了哪些人机交互的原则？（至少三点）

图片内容描述：

支付宝APP的两个界面，第一个界面为语言切换（中文，繁体，English）

第二个界面为修改字体大小的界面

具体详情可以直接打开你的支付宝app自己进入这个界面看~

2005年

简答题

软件成熟度模型

9. 描述软件工程研究所提出的能力成熟度模型（CMM），并详细描述其每一级的名称和特征。（13 分）

解答

[参考](#)

一级为初始级(Initial)：无序的 甚至混乱的

二级为可重复级(Repeatable)：有纪律的

三级为已定义级(Defined)：标准的 一致的模型

四级为已管理级(Managed)：度量的和可预测的

五级为优化级(Optimized)：预防过程和产品缺陷，关注过程持续改进

软件配置管理 + 变更源

10. 什么是软件配置管理（4 分）？描述四个系统基本变更源（4 分）。

解答：

软件配置管理：用技术和管理的指导和监督方法，来标识和说明配置项的功能和物理特征，控制对这些特征的变更，记录和报告变更处理及其实现，并验证与需求规格的一致性

 四个基本变更源

- 新的业务或市场条件
- 新的客户需求
- 企业改组或扩大/缩小规模
- 预算或进度安排的限制

解释几种测试方法

11. 请解释以下和测试有关的概念：集成测试，回归测试， α 测试， β 测试（4 分）

3. 请解释以下和测试有关的概念: 集成测试、回归测试

α 测试、 β 测试

集成测试: ~~也叫组装测试或联合测试。在单元测试的基础上将所有~~

~~模块按照设计要求组装成为系统或子系统, 进行集成测试。~~

又被称为组装测试, 即对程序模块一次性或采用增量的方式组装起来

对系统接口进行正确性检验的测试工作。P₃₂₄

单元测试: (也叫模块测试) 是对程序单元(软件设计的最小单位)进行正确性

检验的测试工作 P₃₂₃

系统测试:

回归测试: 是指修改了旧代码后, 重新进行测试以确认修改没有引入新的
错误或导致其他代码产生错误。

Alpha 测试: 是由一个用户在模拟环境下进行的测试, 也可以是公司内部的用户

在模拟实际操作环境下的受控测试。 α 测试不能由测试人员完成

Beta 测试: 是软件的多个用户在实际使用环境下进行的测试。开发者通常

不在测试现场。Beta 测试不能由程序员或测试人员完成。

2013

解答

✖ 有理数类的实现与测试 (JUnit)

43. (10 分) 数学上, 有理数 (Rational number) 是一个整数 a 和一个非零整数 b 的比, 通常写作 a/b , 故又称作分数。 a 是被除数 (Dividend), b 是除数 (Divisor)。有理数集对加、减、乘、除四则运算是封闭的。

(1) 根据面向对象封装的思想设计一个有理数类, 用以进行有理数的四则运算。写出这个类的完整代码实现。(实现时整数用 `int` 类型表示, 不考虑无穷大的整数和计算超出 `int` 类型范围的情况)

(2) 写出对该类的除法进行单元测试的用例的代码实现。

```
public class Rational{
    private int i;
```

```

private int j;
public Rational(){
    this.i = 1;
    this.j = 1;
}
public Rational(int i,int j){
    this.i = i;
    this.j = j;
}
public void setRational(int i,int j){
    this.i = i;
    this.j = j;
}
public int getI(){
    return this.i;
}
public int getJ(){
    return this.j;
}
public Rational add(Rational r){
    Rational sum = new Rational();
    sum.setRational((int)(i*r.getJ() + j*r.getI()), (int)this.j*r.getJ())
}
public Rational subtract(Rational r){
    pass;
}
public Rational Multiple(Rational r){
    pass;
}
public Rational Division(Rational r){
    pass;
}
public void toString(){
    return i+"/"+j;
}
}

/*
测试代码
*/
public class RationalTest(){
    public static void main(String []args){
        Scanner scanner = new Scanner(System.in);
        Scanner in=new Scanner(System.in); System.out.println("please enter two
number");
        Rational a=new Rational(in.nextInt(),in.nextInt());
        Scanner in=new Scanner(System.in); System.out.println("please enter two
number");
        Rational a=new Rational(in.nextInt(),in.nextInt());

```

```

        Rational result = a.Division(b);
        if(result.getI()!=a.getI()*b.getJ() || result.getJ()!=a.getJ()*b.getI()){
            System.out.println("error");
        }
    }
}

```

使用JUnit设计测试代码

```

//使用JUnit 仿照P294
@RunWith (Value = Parameterized.Class)
public class RationalTester{
    private Rational a;
    private Rational b;
    private Rational result;

    @Parameters
    public static Collection<Rational> getTestParameters(){
        return Array.asList(new Rational[][]{
            // a , b
            {new Rational(1,2),new Rational(3,4),new Rational(4,6)},//测试用例1 , 1/2
            // 3/4 = 4/6
            {new Rational(),new Rational()},//
            {new Rational(),new Rational()},//
            {new Rational(),new Rational()},//
        })
    }

    public ParameterizedTest(Rational a, Rational b,Rational result){
        this.a = a;
        this.b = b;
        this.result = result;
    }

    @Test
    public void testRational(){
        assertEquals(result,a.Division(b));
    }
}

//此外, 还需要重写一下 Rational 的 equals() 方法
bool equals(Rational b){
    return this.getI()==b.getI() && this.getJ()==b.getJ();
}

```

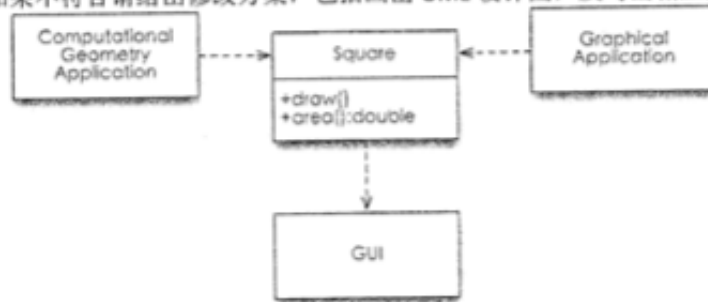
正方形类-长方形

44. (15 分)在某数学软件中，用以下类表示一个长方形。

```
public class Rectangle{
    public double length;
    public double width;
    public double getLength(){ }
    public double getWidth(){ }
    public void setLenth(double l){ }
    public void setWidth(double vv){ }
    public double getArea(){ return length*width; }
}
```

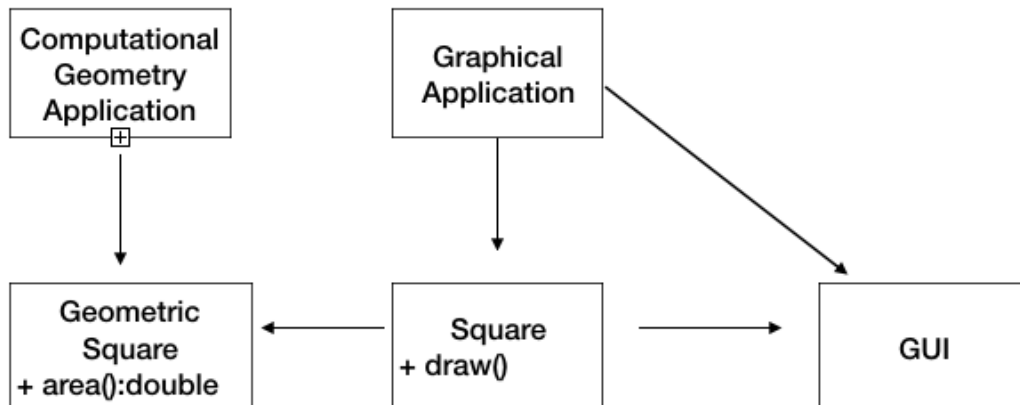
(1) 现在要设计一个正方形类，可以继承自 Rectangle 类么？请说明理由，并给出实现正方形类的代码。

(2) 现在由于需要画出这个正方形，所以需要添加 draw() 方法。该 Square 类分别被几何计算应用类和图形应用类所使用。Square 的 draw() 方法依赖于 GUI 类来实现。为此有人作出如下设计。请问这样的设计符合面向对象的原则么？请给出理由，如果不符合请给出修改方案，包括画出 UML 设计图，及写出相应的代码。



解答

1. 违反LSP，用组合代替继承，见期末B卷
2. 不符合单一职责原则：



2014

解答题

描述软件体系结构分层风格（略）

建立分析类图---创建路单

44. (10 分) 请为下面用例描述建立分析类图。

名称	创建路单
参与者及其目标	调度室：生成路单
触发条件	选择用车计划（不选则为默认计划），开始新建路单
前置条件	选择用车计划（不选则为默认计划）
后置条件	如果该用车计划的所有需求车辆都已经开出了路单，则该用车 设为已执行状态。
正常流程	1、 用户选择用车计划/请求添加该计划的路单 2、 系统允许用户填写基本路单信息 3、 用户请求选择车辆 4、 系统返回待派车辆列表信息 5、 用户选中车辆（多选） 6、 系统记录被选中车辆 7、 用户请求输入路程信息 8、 系统允许用户输入路程信息 9、 用户提交，生成路单 10、系统生成路单附带出门证（碰） 11、系统打印路单条
分支流程	5a 备选车辆不够，可以手工添加其他车辆进入* 9a 如果是长途路单，则不直接生成，而是生成长途路单并请 1 报至领导处，审批通过后，继续 8
异常流程	1a 不选则计入为默认计划中
相关用例	Include: M4 新建出门证

解答：来自学长（P100）

①识别候选类:1.用户 2.用车计划 3.添加路单请求 4.基本路单信息 5.选择车辆请求
6.车辆列表 7.车辆 8.输入路程信息请求 9.路单 10.出门证
11.路单条 12.路单创建

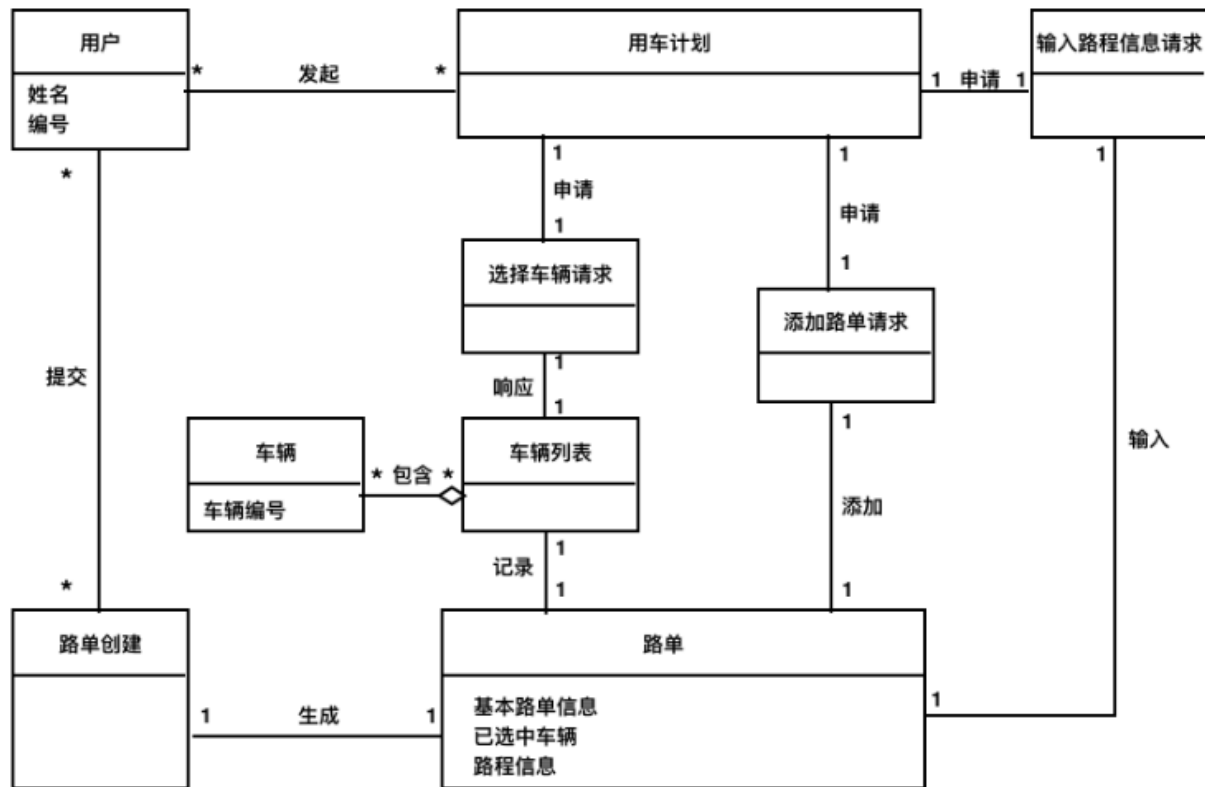
②筛选候选类，确定概念类:

- | | |
|---------------------------|--------------|
| 1.用户(姓名, 编号) | 2.用车计划 |
| 3.添加路单请求 | 4.选择车辆请求 |
| 5.车辆列表 | 6.车辆(车辆, 编号) |
| 7.输入路程信息请求 | |
| 8.路单(基本路单信息, 已选中车辆, 路程信息) | |
| 9.路单创建 | |

③识别关联: 1.车辆列表 包含 车辆
2.用户 发起 用车计划
3.路单 记录 已选中车辆和路程信息

④确定重要属性: 见步骤2括号中的内容

可得分析类图如下:



分析代码质量不高的地方并改进 -- Department类

45. 找出下面代码中质量不高的地方，详细说明其问题，并进行改进。

```

public class Department {
    private List<Employee> employeeList;

    .....
    public Employee addEmployee (int employeeType, String name){
        Employee emp;
        switch(employeeTYPE){
            case EMPLOYEE.SALARIED: emp= new SalariedEmployee(name);break;
            case EMPLOYEE-HOUR: emp= new HourlyEmployee(name); break;
            caseEMPLOYEE.COMMISSION:emp=newComrmissionedEmployee(name); break;
            Case default: emp= new Employee(name);
        } emp.setDepartment(this);
        employeeList-add(emp);
        update();
    }
    .....
}
  
```

解答：

代码问题：违反开闭原则；代码灵活性和可扩展性较低；

🇨🇳 <简单工厂>改进

```

//可以将判断语句下沉到工厂类中
public class Department{
    private List<Employee> emplyeeList;
    public Department(){
  
```

```

    }
    public Employee addEmployee(int employeeType,String name){
        SimpleFactory factory = new simpleFactory();
        Employee employee = factory.createEmployee(employeeType ,name);
        employee.setDepartment(this);
        employeeList.add(employee);
        return employee;
    }
}

public class simpleFactory{
    public Employee createEmployee(int employeeType,String name){
        Employee employee;
        switch(employeeType){
            case EMPLOYEE.SALARIED :employee = new SalariedEmployee(name);break;
            case EMPLOYEE-HOUR: emp= new HourlyEmployee(name); break;
            case EMPLOYEE.COMMISSION:emp=new CommissionedEmployee(name); break;
            default: emp= new Employee(name);
        }
        return employee;
    }
}

```

```

//还可以将判断上抛
//谁调用 addEmployee() 谁判断
public interface Employee{}
public class SalariedEmployee implements Employee{

}

public class HourlyEmployee implements Employee{}
public class CommissionedEmployee implements Employee{}

public class Department{
    private List<Employee>employeeList;
    public Department(){
    }
    public Employee addEmployee(Employee employee){
        employee.setDepartment(this);
        employeeList.add(employee);
        return employee;
    }
}

```

2015

名词解释：

1. 软件工程
2. 软件需求

解答题

图示面向对象体系结构风格，优缺点（略）

建立分析类图 -- 自驾游路线设计

ID	2.1	名称	创建路线
参与者	自助游游客，目标是在获得充分信息的前提下完成一次旅游路线的设计。		
触发条件	自助游游客希望设计一条旅游路线		
前置条件	自助游游客通过了系统验证		
后置条件	保存自助游游客设计的路线		
正常流程	1. 自助游游客请求进行路线设计； 2. 系统展示日程表，帮助游客按照日程表逐一设计路线； 3. 自助游游客选择一个日程，向路线添加新的一天； 4. 系统在日程中新增一天，并把这天作为当前编辑日； 5. 自助游游客向当前编辑日添加新项目。项目包括景点、酒店和交通； 6. 系统向用户展示请求的可用项目的列表。项目的描述参见数据模型【考试中可以自行设计】。 7. 自助游游客在列表中检索并选定项目； 8. 系统把用户选定的项目添加进日程； 9. 用户重复步骤 2-8，知道完成路线设计； 10. 用户完成路线设计。系统保存这条路线，添加路线到自助游游客的路线列表。		
扩展流程	3a、删除日程。 1. 自助游游客请求从行程中删除某日程； 2. 系统提示当天日程将被全部清空； 4-8a、删除项目 1. 自助游游客从行程的当前编辑日中删除某项目； 2. 系统从当前编辑日中删除该项目。		

解答：见「软件工程强化与冲刺」

代码质量不高的地方并改进 -- init() 方法

```
class A{
    private:
        FinancialReport fr;
        WeatherData wd;
        int totalCount;
    public:
        A();
        void init(){
            //
            fr = new FinalcialReport();
            fr.setRatio(5);
            fr.setYear("2010");

            //
            w = new WeatherData();
        }
}
```



```

        w.setCity("NanJing");
        w.setCode("210093");

        //
        totalCount = 0;
    }
}

```

解答：

时间内聚

改进1: 拆分类

```

class FinalcialReport{
    private:
        int ratio;
        String year
    public:
        void setRatio(int ratio){
            this->ratio = ratio;
        }
        void setYear(String year){
            this->year = year;
        }
        void init(){
            this->setRatio(5);
            this->setYear("2010");
        }
}

class WeatherData{
    private:
        String city;
        String code;
    public:
        int setCity(String city){
            this->city = city;
        }
        int setCode(String code){
            this->code = code;
        }
        void init(){
            this->setCity("NanJing");
            this->setCode("210093");
        }
}

class A{
    private:
        FinancialReport fr;
        WeatherData wd;
}

```

```

    int totalCount;
public:
    void init(){
        this->totalCount = 0;
        fr.init();
        wd.init();
    }
}

```

改进2: 拆分方法

```

class A{
private:
    FinancialReport fr;
    WeatherData wd;
    int totalCount;
public:
    A();
    void initFinancialReport(){
        fr = new FinalcialReport();
        fr.setRatio(5);
        fr.setYear("2010");
    };
    void initWeatherData(){
        w = new WeatherData();
        w.setCity("NanJing");
        w.setCode("210093");
    }
    void initTotalCount(){
        totalCount = 0;
    }
}

```

修改不足 -- 多个compare

八、下列方法要用于一个关键系统，但其可靠还有不足。请找出其中不足的地方，并进行改进。（10分）

```
Public Boolean compare(Double leftOperand, Double rightOperand, COMPARISON_SIGN
comparisonSign ) {
    boolean result = true;
    switch(comparisonSign){
    case equal: {
        if (leftOperand !=rightOperand) {
            result = false;
        }
        break;
    }
    case not_equal: {
        if(leftOperand == rightOperand) {
            result = false;
        }
        break;
    }
    case large_equal: {
        if(Math.abs(leftOperand)<Math.abs(rightOperand)){
            result = false;
        }
        break;
    }
    case large: {
        if(Math.abs(leftOperand)<=Math.abs(rightOperand)){
            result = false ;
        }
        break;
    }
    case less_equal:{
        if(leftOperand>rightOperand){
            result = false;
        }
        break;
    }
    case less: {
        if(Math.abs(leftOperand)>=Math.abs(rightOperand)){
            result = false;
        }
        break;
    }
    case none: {
        break;
    }
    }
    return result;}
}
```

解答：用策略模式

```
interface Compare{
    public boolean compare(Double leftOperand, Double rightOperand);
}
public class Equal implements Compare{
    public boolean compare(Double leftOperand, Double rightOperand){
        if(leftOperand != rightOperand){
            return false;
        }
        return true;
    }
}
```

```

}
public class NotEqual implements Compare{
    public boolean compare(Double leftOperand, Double rightOperand){
        ...
    }
}
public class LargeEqual implements Compare{
    public boolean compare(Double leftOperand, Double rightOperand){
        ...
    }
}
public class Large implements Compare{
    public boolean compare(Double leftOperand, Double rightOperand){
        ...
    }
}
public class LessEqual implements Compare{
    public boolean compare(Double leftOperand, Double rightOperand){
        ...
    }
}
public class Less implements Compare{
    public boolean compare(Double leftOperand, Double rightOperand){
        ...
    }
}

public class Context{
    private Compare compare;
    public void setCompare(Compare com){
        this.compare = com;
    }
    boolean getCompareRes(Double leftOperand, Double rightOperand){
        return compare.compare(leftOperand, rightOperand);
    }
}

public class Client{
    Equal equal = new Equal();
    NotEqual not_equal = new NotEqual();
    LargeEqual large_equal = new LargeEqual();
    Large large = new Large();
    LessEqual less_equal = LessEqual();
    Less less = new Less();

    Context context = new Context();
    context.setCompare(equal);
    //context.setCompare(not_equal);
    //context.setCompare(large_equal);
    // ...
}

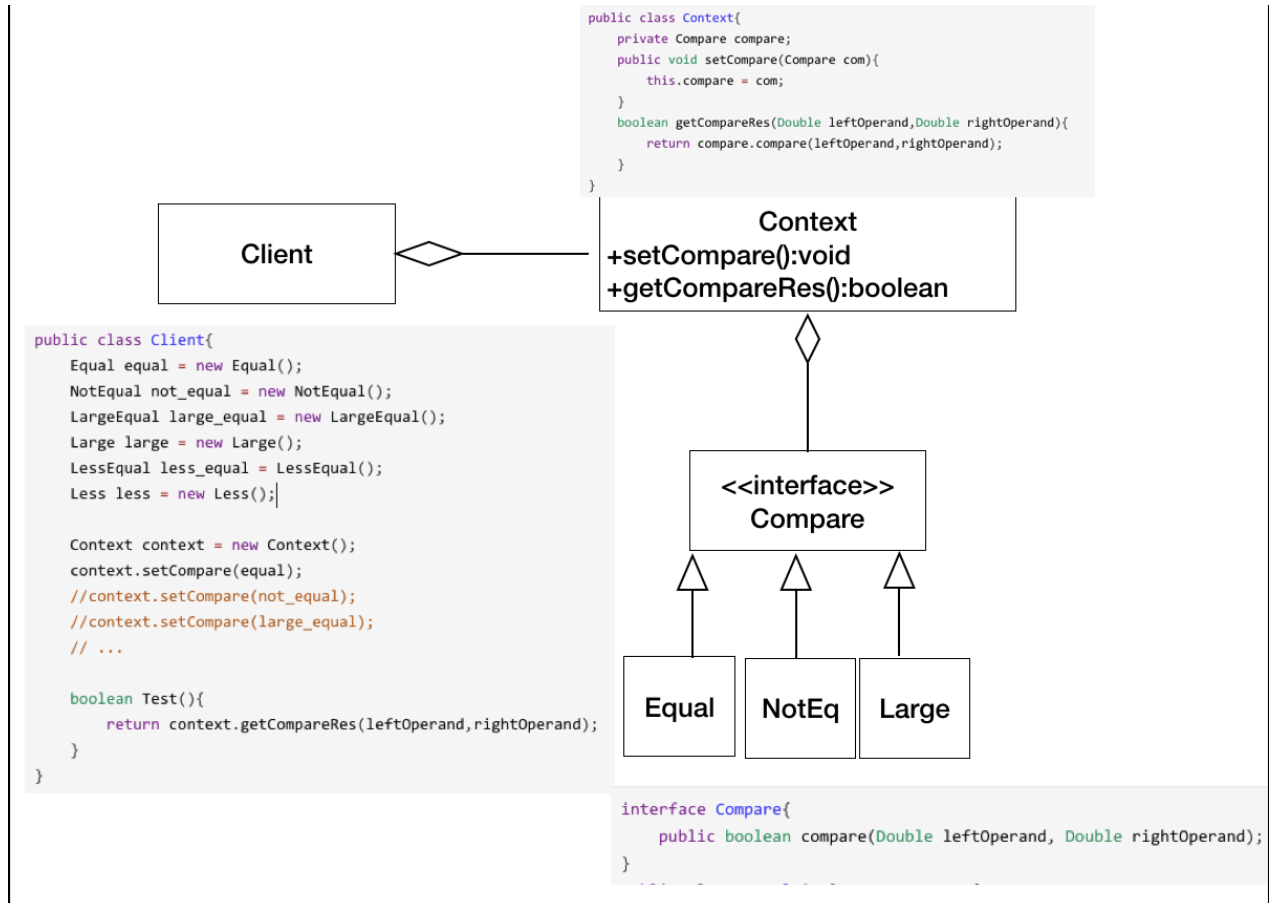
```

```

boolean Test(){
    return context.getCompareRes(leftOperand,rightOperand);
}
}

```

类图：



2016

质量保障的三种手段

评审，测试，质量度量

判断需求类型

四、下列需求属于那种类型？（5分）

（1）在系统使用 6 个月后，销售成本应该降低 25%；

（2）系统应该能够存储 5 年的销售数据；

（3）销售人员可以使用系统指定销售订单；

（4）系统应该可以在 iOS 和 Android 两种主流移动平台端使用；

（5）在销售人员输入 ID 请求查询客户信息时，系统应该显示客户的名称，单位地址、联系方式、行业。

解答：

P76

1. 功能
2. 性能
3. 功能
4. 约束
5. 功能

ATM取款用例

解答：

参与者	存、取款人
触发条件	将银行卡插入 ATM 机
前置条件	该卡被 ATM 机识别和授权
后置条件	存、取记录，更新数据，打印凭条
正常流程	1. 输入密码 2. 存款/取款选择 3. 输入金额/放置现金 4. 查看余额（可跳过） 5. 退出，拔出卡片 6. 打印凭条

上面的答案过于简单

解答二

取款处理

ID:WithdrawProcess

参与人:用户(取款人), 目标是能够尽可能快速地完成取款业务

触发条件:用户将银行卡插入卡槽

前置条件:所插入磁卡是本银行发行且是有效状态的银行卡

后置条件:更新账户信息, 准确读出取款现金至取款槽

正常流程:

- 1.系统对插入卡槽的银行卡进行识别
- 2.系统显示用户操作界面
- 3.用户选择取款业务, 系统显示卡上余额
- 4.用户输入取款金额, 系统更新账户信息, 并读出相应现金
- 5.用户选择退卡, 系统退回主界面, 并动态更新银行基金宣传信息

扩展流程:

- 1a.无效磁卡, 吐出银行卡, 并提示错误信息
- 3a.非法字符, 系统提示错误并拒绝输入
- 3b.用户输入金额超过余额, 系统提示并拒绝提款
- 4a.用户提款完成后, 系统语音提示用户退卡, 防止丢失

特殊需求:

- 1.用户所有操作应在3s内得到响应
- 2.初次使用ATM取款的用户也能快速完成取款业务。

违反原则-改进代码 --- employee类

六、下面是 Employee 的设计示意, 请为 Employee 类的设计违反了哪条面向对象的设计原则? 会产生何种后果? 应该如何改进? (10 分)

```
public class Employee{
    private string EmployeeName { public get,set;
    private int EmployeeNo { public get,set;
    public void insert() {
        //Database Logic written here    }
    public static Employee findById(int id) {
        //Database Logic written here    }
    public void GenerateReport(){
        //Set report formatting
    }
}
```

解答

违反单一职责原则, 内聚性不够, 影响修改和扩展时的灵活性。

将insert(), findById(), GenerateReport()分别放在单独的类中

用契约式/防御式 改写代码 --- Deposit类

七、 下面是银行金融系统中账户类的取款方法 `deposit` 实现，它在可靠性上存在哪些问题？请应用契约式编程或者防御性编程方法改写该方法的代码（15 分）。

```
Class BankAccount {
    private Double balance; //账户数额
    private Double dailyBalance; //当日可用数额
    ....
    /*
     *ATM 机取款方法
     *基本功能是取出 100 元的整数倍人民币
     *但是一次只能取不超过 3000 元
     *一天取钱总数不能超过 20000 元
     *账户不能透支，最低应该有 1 元留在账户上
     */
    Public void deposit(Double num) {
    try {
        exportCash(num)
        balance -=num;
        dailyBalance-=num;
    } catch(ExportCashException e) {
        ExportCashExceptionHandler(e);
    }
    }
    /**
     *ATM 机控制出钞口吐出现金纸币
     *吐出 100 元的整数倍人民币
     *如果失败抛出 ExportCashException 异常
     */
    public void exportCash(Dauble num) throw ExportCashException {
        ....
    }
}
```

解答

```
//断言式
public void deposit(Double num){
    //前置条件
    assert (num<= 3000 && num%100==0):("error");
    try{
        ..
    }catch{
        ..
    }
    //后置条件
    assert (dailyBalance >= 0&&balance >= 1):("error");
}
```

2017

名词解释

软件工程

持续集成

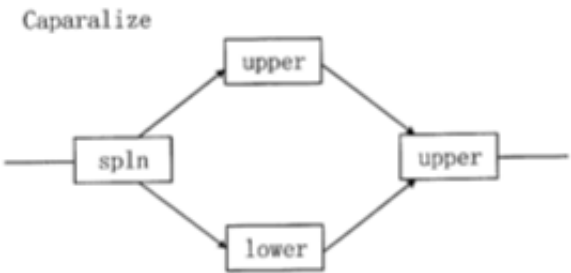
需求题

需求的三个层次

从三个层次出发，描述图书管理系统中的需求

体系结构 -- 大小写转换

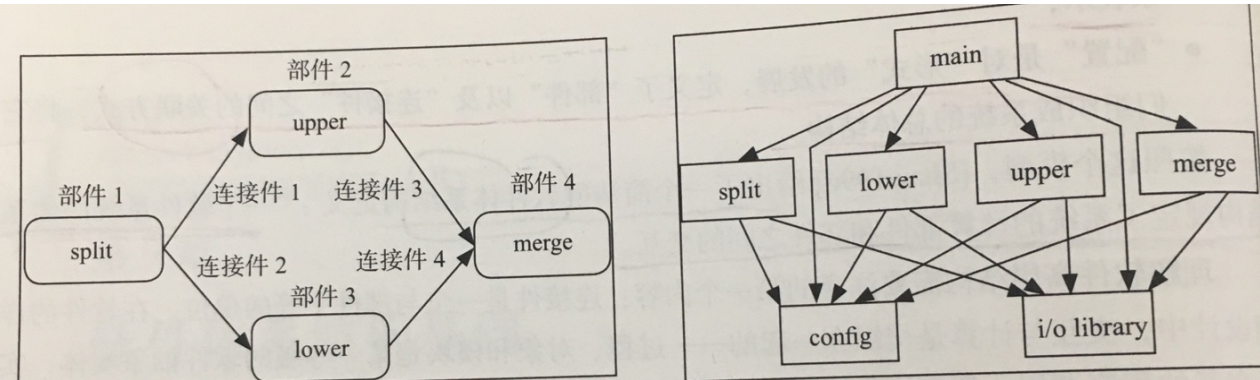
将一个字符串读入，修改字符串使其呈现大小写交替出现，然后输出。如 I love this game
-> I Love This Game
下图是系统的逻辑设计，将字符串先分解，再处理大小写转换，最后合并。



请根据某种体系结构风格，给出系统的物理设计模块依赖管理图并解释相应模块的职责。

解答： P150

主程序/子程序风格



类的职责与封装 -- person类

从职责和封装的角度，下列这个类的设计如何？如果号，解释里有，如果不好，解释理由并改正。

```
Public class Person {
    String name;
    Public getAge(){}
```

解答

总感觉者答案写的有点牵强

方法与成员变量不匹配,导致内聚过低。不满足信息内聚,应该最大化的封装信息和行为。

```
Public class Person {  
    Private String name;
```

279

踏实、勤奋、认真

```
    Private String age;  
    Public setName() {};  
    Public getName() {};  
    Public setAge() {};  
    Public getAge() {};  
}
```

面向对象 -- 计算个人所得税

下列是计算雇员所得税代码,请从交互和协作的角度分析代码是否合理

```
Public class Employee{
```

275

踏实、勤奋、认真

```
    Double income;  
    Double getTax(){  
        Return income*tax.getTaxrate();  
    }  
}  
Public class Tax{  
    Double taxrate;  
    Double getTaxrate(){  
        Return taxrate;  
    }  
}
```

解答：违反迪米特

getTex()逻辑上并不属于Employee的方法，违反迪米特法则，并造成隐式访问

```
public class Employee{
    private Double income;
    public Double getIncome(){
        return income;
    }
}
public class Tax{
    private Double taxRate;
    private Employee employee;
    public Double getTaxRate(){
        return raxRate;
    }
    public Double getTax(){
        Employee employee = new Employee();
        return empolyee.getIncome*taxRate;
    }
}
```

```
public class Employee{
    private Double income;
    public Double getIncome(){
        return income;
    }
    public getTax(){
        Tax tax = new Tax();
        return Tax.calculateTax(income);
    }
}
public class Tax{
    private Double taxRate;
    public Double getTaxRate(){
        return raxRate;
    }
    public Double calculateTax(Double income){
        return income * taxTate;
    }
}
```

✗感觉上面两个代码都不太好，怎么改进？

面向对象 -- 网络选课系统

下列是网络选课系统的部分代码，请从面向对象角度使用多态对以下代码进行合理修改。

```
processCmd (int cmdID) {  
    switch (cmdID) {  
        case1: addCourse ();break;  
        case2: removeCourse ();break;  
        ...  
    }  
}
```

解答

```
//将判断上抛  
interface Command{  
    public void handle(){}  
}  
public class addCourse{  
    public void handle(){  
  
    }  
}  
public class removeCourse{  
    public void handle(){}  
}  
public Test{  
    processCmd(Command cmd){  
        cmd.handle();  
    }  
}
```

```
//将判断下沉  
//利用工厂模式，将判断放到工厂中
```

消除代码重复 -- getTotalSum

十、构造题。如何消除下面代码的重复

```
Private Double getTotalSum(List amounts) {
    Double totalToPay = 0.00;
    Iterator amountsIterator = amounts.iterator();
    While(amountsIterator.hasNext()){
        Amount amount = (Amount) amountsIterator.next();
        if(!cancelStatuses.contains(amount.getStatus())) {
            totalToPay += amount.doubleValue();
        }
    }
    return new Double(totalToPay);
}

Private Double getTotalSumExcludeCancelAmount(List amounts) {
    Double totalToPay = 0.00;
    Iterator amountsIterator = amounts.iterator();
    While(amountsIterator.hasNext()){
        Amount amount = (Amount) amountsIterator.next();
        If (!amount.getIsToCancel()) { // Additional condition
```

```
        Comparing to the first method
        If (!cancelStatuses.contains(amounts.getStatus())){
            totalToPay += amount.doubleValue();
        }
    }
    Return new Double(Topay);
}
```

解答1

```
//设置标志位来判断需要输出哪个结果
private Double getSum(List amounts,int flag){
    Double totalToPay = 0.00;
    Double totalToPayExcludeCancel = 0.00;
    Iterator amountIterator = amounts.iterator();
    while(amountIterator.hasNext()){
        Amount amount = (Amount)amountIterator.next();
        if(!cancelStatuses.contains(amount.getStatus())){
            totalToPay += amount.doubleValue();
            if(!amount.getIsToCancel()){
                totalToPayExcludeCancel += amount.doubleValue();
            }
        }
    }
    if(flag)return Double(totalToPay);
    else return Double(totalToPayExcludeCancel);
}
```

解答2

下面的答案写的啥玩意，看不懂

```

    }
    private Double getTotalSumExcludeCanaelAmount(List amounts){

        double totalToPay = 0;
        Iterator amountsIterator = amounts.iterator();
        while (amountsIterator.hasNext()){
            Amount amount = (Amount)amountsIterator.next();
            //需要排除
            if (!amount.getIsToCancel()) {
                totalToPay += getpay(amount);
            }
            //如果不需要去掉if
        }
        return new Double(totalToPay);
    }
    private double getpay(Amount amount){
        String cancelstatuses = "aassdd";
        if (!cancelstatuses.contains(amount.getStatus())){
            return amount.dubleValue();
        }
        return 0.0;
    }
}

```

测试题

什么是黑盒测试

黑盒测试的方法

等价类划分，边界值分析，决策表，状态转换

2018

问答题

科学与工程的区别

集成测试，单元测试，集成测试与单元测试用例的区别

功能性需求与非功能性需求

四个软件体系结构视角

质量模型的可用性

× 房屋评估系统的合理性

1、一个购房评估系统，一个类中含有两个功能，一个是输入存款、月工资、月花销、想要购房面积..... 完后得出可以购房的面积和首付款，另一个功能是根据面积和首都得出一个各小区的房价列表，供用户参考，问这种设计合理吗？不合理的话画出设计类图并写出类的定义（含属性和方法）

解答

✗计算税费

2、一个计算税费的问题，根据什么一堆经济的指标什么印花税、公证费、契税、委托办理手续费、
房屋买卖手续费啥的，计算出税费，各地的计算方法不同，要求能够灵活扩展，按照一种设计模式设计，画出设计类图并写出关键接口定义。

解答

2013备用卷

面向对象 -- 计算点之间的距离与方向

```
public class Point{
    public double x;
    public double y;
}
public class PointUtility{
    public static double calculateDistance(Point a, Point b){
        .....
    }
    public static double calculateDirection(Point a, Point b){
        ...
    }
}
```

这样的设计，是否符合面向对象设计的原则？如果符合，给出理由，如果不符，同样给出理由并给出修改方案。

解答：没有集中信息与行为

不符合，没有集中信息与行为，内聚性低 P248

```
public class Point{
    private double x;
    private double y;
    public double calculateDistance(Point b){}
    public double calculateDirection(Point c){}
}
```

✗智能家居系统

44.(15 分) 某智能家居软件中，有一系列各式各样的按钮（ButtonA, ButtonB, ButtonC），还有一系列种类的门（DoorA, DoorB, DoorC）。以下是部分残缺的代码：

```
public class Door{
    public void open(){
        door.open();
    }
}
public class Button{
    public void turnOn{ door.open();
}
private Door door;
```

(1)如果想自由的设置由某个 Button 控制某个灯。并且希望能够灵活的更改，该如何设计？画出 UML 设计图，写出相应的代码。

(2)如果我想让一个 Button 同时开关多盏灯，如何设计？画出 UML 设计图，写出相应的代码。

(3)如果系统中除了灯还有别的家电需要 Button 来控制，那么怎么设计？画出 UML 设计图，写出相应的代码。

解答

不是控制「门」吗，怎么到了题目成了控制灯？？？

1. 每个Button实例控制一盏门，抽象Door，使得Door可灵活更改。二者直接依赖关系

```
public class Button{
    Door door;
    public Button(Door d){
        door = d;
    }
    public void turnON(){
        door.open();
    }
    public void turnOFF(){
        door.close();
    }
}
public interface Door{
    public void open();
    public void close();
}
```



```

}
public class DoorA implements Door{

}

public class DoorB implements Door{

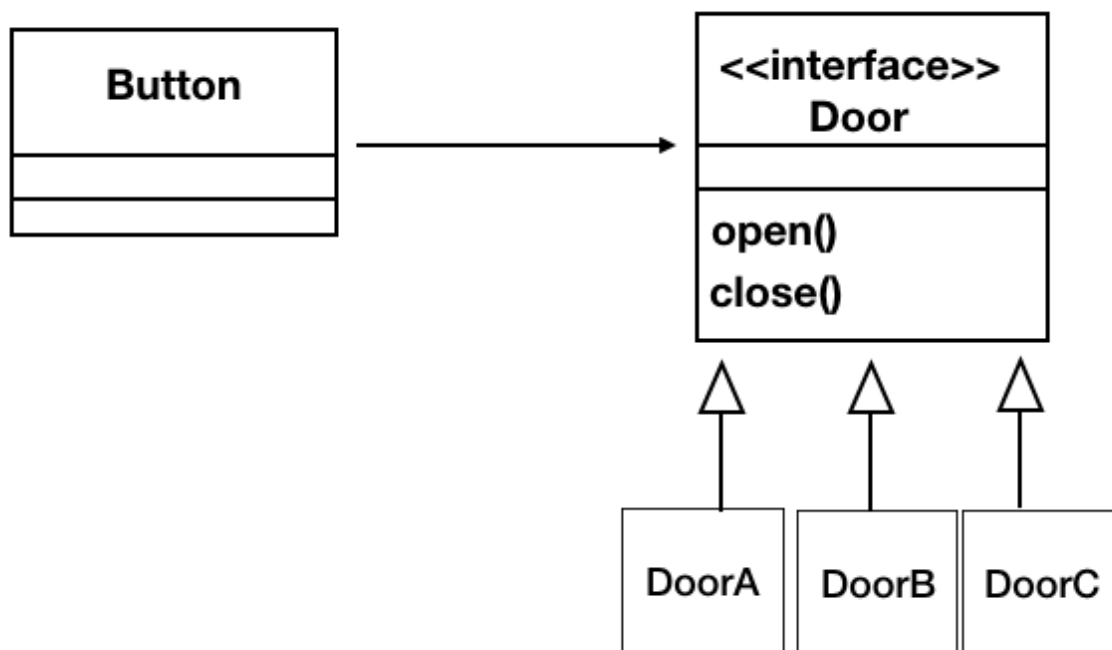
}

public class DoorC implements Door{

}

public class TestClient{
    public static void main(string []args){
        Door doora = new DoorA();
        Button buttona = new Button(doora);
        buttona.turnON();
    }
}

```



2. 一个Button控制多个灯

```

import java.util.ArrayList;
public class Button{
    private ArrayList<Door> doorList = new ArrayList<>();
    public Button(ArrayList<List> doorlist{
        doorList = doorlist;
    }
    public void turnON(){

```

```

        for(door:doorList){
            door.open
        }
    }
    public void turnOFF(){
        for(door:doorList){
            door.close();
        }
    }
    public void addDoor(Door d){
        doorList.add(d);
    }
    public boolean deleteDoor(Door d){

    }
}
public interface Door{
    public void open();
    public void close();
}
public class DoorA implements Door{

}
public class DoorB implements Door{

}
public class DoorC implements Door{

}

```

3. 需要控制多种设备，把包括灯在内的所有设备抽象为Device接口，Button只持有Device接口，不区分具体是什么设备

```

import java.util.LinkedList;
public class Button{
    LinkedList<Device> deviceList = new LinkedList<>();
    public Button(LinkedList deviceList){
        deviceList = deviceList;
    }
    public void turnON(){
        door.open();
    }
    public void turnOFF(){
        door.close();
    }
    public void addDevice(Device d){
        deviceList.add(d);
    }
    public boolean deleteDevice(Device d){

```

```
        deviceList.remove(d);
    }
}
public interface Device{
    public void open();
    public void close();
}
public class Door implements Device{
    public Door(){
        super();
    }
}
public class DoorA extends Door{

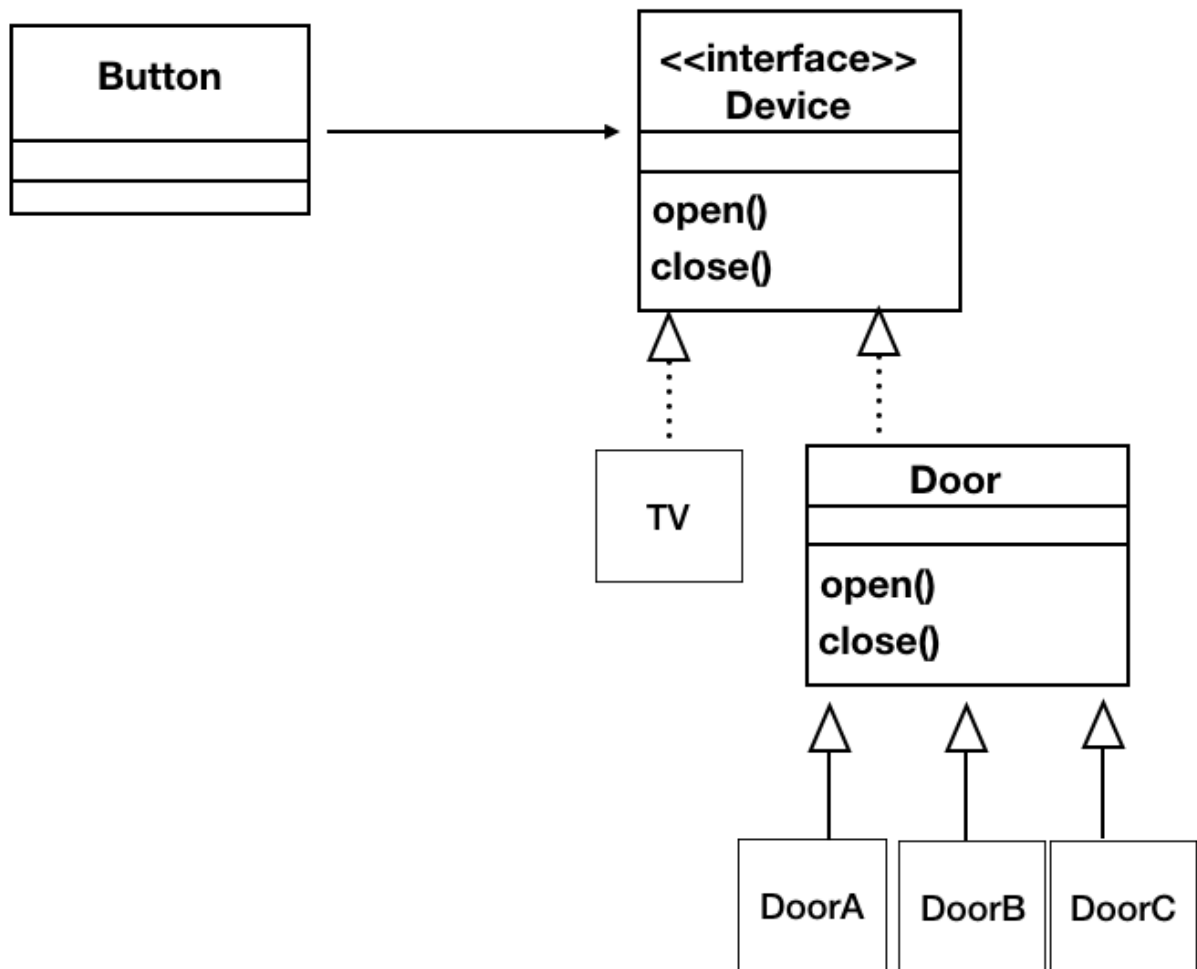
}
public class DoorB extends Door{

}
public class DoorC extends Door{

}
public class TV implements Device{

}
public class AirConditioner impenents Device{

}
```



2019

简答题

生命周期模型

功能需求与非功能需求

软件开发不同阶段的质量保证手段

体系结构的四个视角

黑盒白盒差异

迪米特法则 --- sales类

1. 迪米特: sales - salelist - saleslinetiem - commodity (10 分) 书上原题
(1) 违反了什么面向对象设计原则并解释这个原则 (3 分)
(2) 书上代码写出来 (4 分)
(3) 书上顺序图画出来 (3 分)

解答: P233

设计模式 --- 飞机类

2. 飞机类型: A、B、C、D (这是某本设计模式书上原题)
起飞类型: TakeOff1 TakeOff2
飞行类型: Flight1 Flight2
(1) 策略模式 uml 图
(2) 策略模式应用时的优缺点有哪些? 和其他设计模式比较?

解答

UML图类似于 P268

策略模式的优缺点:

1. 优点:
 - 避免多重选择语句
 - 可以动态选择不同的策略
2. 缺点:
 - 用户必须事先知道所有的策略
 - 会创建出较多的对象

附录 -- 遗留未解决的问题

2017年个人所得税问题

下列是计算雇员所得税代码,请从交互和协作的角度分析代码是否合理

```
Public class Employee{
```

275

踏实、勤奋、认真

```
    Double income;  
    Double getTax(){  
        Return income*tax.getTaxrate();  
    }  
}  
Public class Tax{  
    Double taxrate;  
    Double getTaxrate(){  
        Return taxrate;  
    }  
}
```

解答：违反迪米特

getTex()逻辑上并不属于Employee的方法，违反迪米特法则，并造成隐式访问

```
public class Employee{  
    private Double income;  
    public Double getIncome(){  
        return income;  
    }  
}  
public class Tax{  
    private Double taxRate;  
    private Employee employee;  
    public Double getTaxRate(){  
        return raxRate;  
    }  
    public Double getTax(){  
        Employee employee = new Employee();  
        return empolyee.getIncome*taxRate;  
    }  
}
```

```

public class Employee{
    private Double income;
    public Double getIncome(){
        return income;
    }
    public getTax(){
        Tax tax = new Tax();
        return Tax.calculateTax(income);
    }
}
public class Tax{
    private Double taxRate;
    public Double getTaxRate(){
        return raxRate;
    }
    public Double calculateTax(Double income){
        return income * taxTate;
    }
}

```

✗ 感觉上面两个代码都不太好，怎么改进？

✗ 怎么修改更合理，Employee和Tax类应该是怎样的关系？

✓ 应用策略模式

```

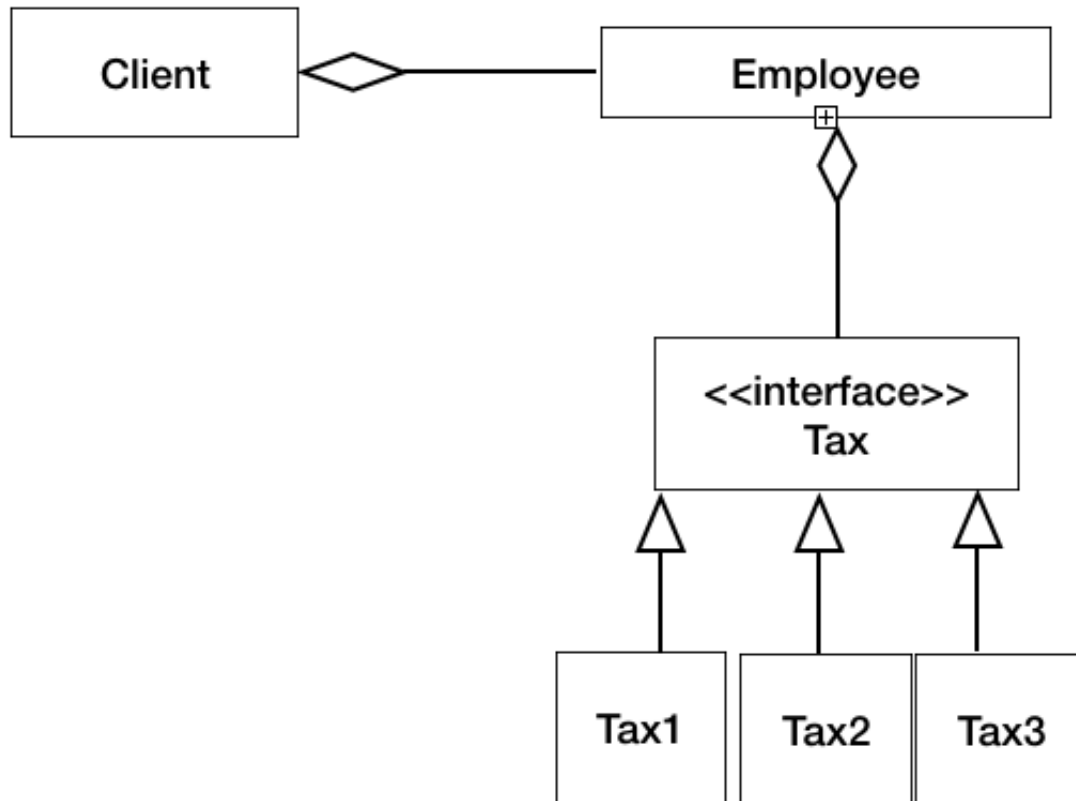
public class Employee{
    private Double income;
    private Tax tax;           //雇员持有一个策略接口
    public Double getIncome(){
        return income;
    }
    public void setTax(ConcreteTax cTax){//设置策略
        tax = cTax;
    }
    public Double getTax(){//调用策略的计算
        return tax.calculateTax(income);
    }
}
public interface Tax{ //策略接口
    public void calculateTax(int income);
}
public class ConcreteTax implements Tax{ // 具体策略
    private Double taxRate;
    public void calculateTax(int income){
        return income*taxRate;
    }
}
public class Client{

```

```

private Employee employee;
ConcreteTax cTax = ConcreteTex();
employee.setTax(cTax); //为雇员设置策略
System.out.print(employee.getTax()); //计算结果
}

```



几个软件测试题代码

Kawata 同学希望测试 Akagi 同学设计的类

```

Public class MyStackextendsVector{
    Public void push(Objectelement){
        insertElementAt(element,0);
    }
    Public Object pop(){
        Object result=firstElement();
        removeElementAt(0);
        Return result;
    }
}

```

- 1)完成功能测试的测试用例的设计，说明思路。
- 2)给出相应的测试代码。

解答

参考答案（不太靠谱）

(1)

ID	输入	预期输出
1	push("a"),push("b")	a
2	push(1),push(2)	1

```
(2)class Test {
    Public static void main() {
        Mystack s = new Mystack();
        s.push("a");
        s.push("b");
        s.pop();
        s.show();
    }
}
```

✗怎么设计测试用例和测试代码

2013年备用 按钮-灯泡-门

44.(15 分) 某智能家居软件中，有一系列各式各样的按钮（ButtonA, ButtonB, ButtonC），还有一系列种类的灯（DoorA, DoorB, DoorC）。以下是部分残缺的代码：

```
public class Door{
    public void open(){
        door.open();
    }
}
public class Button{
    public void turnOn{ door.open();
    }
    private Door door;
```

- (1)如果想自由的设置由某个 Button 控制某个灯。并且希望能够灵活的更改，该如何设计？画出 UML 设计图，写出相应的代码。
- (2)如果我想让一个 Button 同时开关多盏灯，如何设计？画出 UML 设计图，写出相应的代码。
- (3)如果系统中除了灯还有别的家电需要 Button 来控制，那么怎么设计？画出 UML 设计图，写出相应的代码。

✗这是个啥，这几个乱七八糟的类怎么协作？

2018房屋评估系统

1、一个购房评估系统，一个类中含有两个功能，一个是输入存款、月工资、月花销、想要购房面积..... 完后得出可以购房的面积和首付款，另一个功能是根据面积和首都得出一个各小区的房价列表，供用户参考，问这种设计合理吗？不合理的话画出设计类图并写出类的定义（含属性和方法）

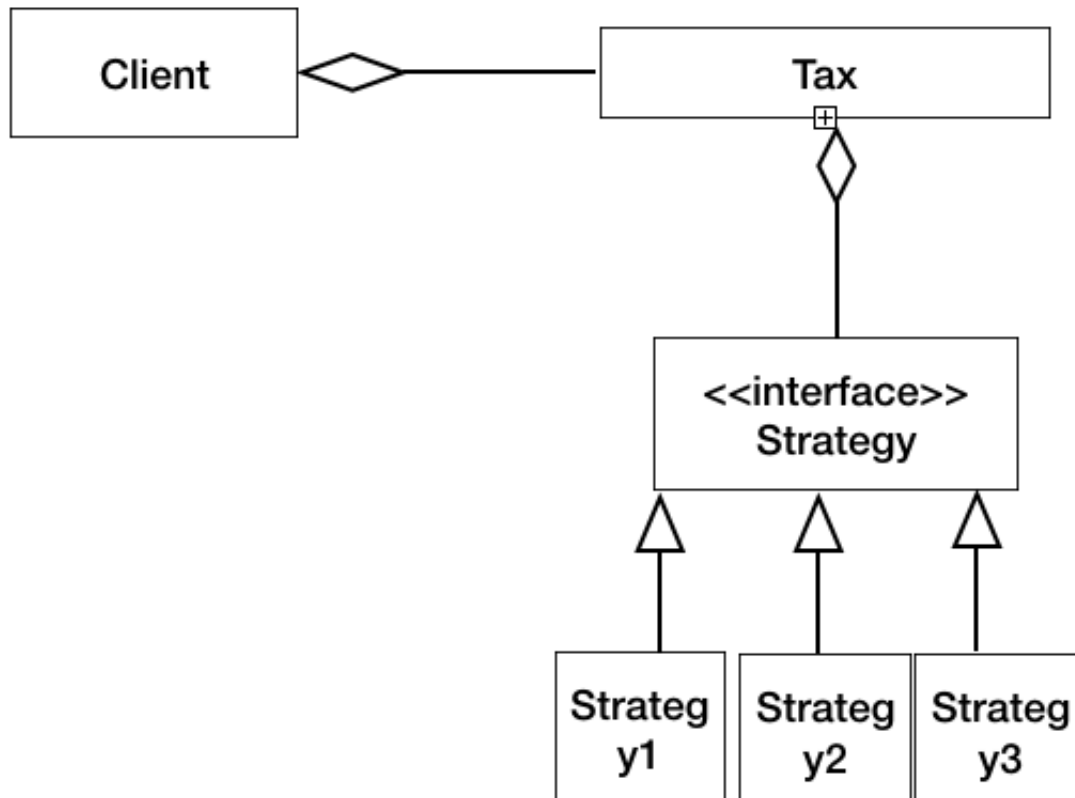
✗怎么设计类，类之间的关系应该是怎么样的？

2018 计算税费问题

2、一个计算税费的问题，根据什么一堆经济的指标什么印花税、公证费、契税、委托办理手续费、

房屋买卖手续费啥的，计算出税费，各地的计算方法不同，要求能够灵活扩展，按照一种设计模式设计，画出设计类图并写出关键接口定义。

这是我画的类图



✗ 印花税，公正税，契税，委托办理手续费，房屋买卖手续费。。 这些应该作为哪个类的属性？？